Winning the 2025 FIT5222 Flatland Challenge

Sean Murphy
smur0055@student.monash.edu

Abstract

This report details the design and implementation of a planner for the Flatland Challenge, a multi-agent pathfinding (MAPF) problem set in complex railway networks. The challenge spans three tasks of increasing complexity: single-agent pathfinding, multi-agent scheduling with static obstacles, and a full-scale multi-agent coordination problem with deadlines and dynamic malfunctions. The solution employs foundational algorithms for the initial stages, using A search for optimal single-agent routing and Safe Interval Path Planning (SIPP) to manage time-dependent constraints. For the final challenge, a scalable metaheuristic framework based on Large Neighborhood Search (LNS) was developed. This framework integrates Prioritised Planning (PP) with a reverse Dijkstra heuristic, adaptive neighborhood selection, and Minimum Communication Policies (MCP) to robustly handle agent malfunctions. The implemented system successfully solved all test instances, achieving a final f score of 105.52% that outperformed the advanced staff baseline and secured first place in the competition. This work demonstrates that a hybrid approach, balancing optimal low-level planners with a high-level heuristic search, is highly effective for solving large-scale, dynamic MAPF problems.*

1. Introduction

Multi-Agent Pathfinding (MAPF) is a core problem in Artificial Intelligence, with applications in railway scheduling, warehouse robotics, and autonomous traffic systems. The Flatland Challenge provides a benchmark for MAPF in railway networks, requiring algorithms to coordinate multiple trains efficiently and safely under spatial and temporal constraints. Agents must reach their destinations without collisions while also handling realistic complications such as shared tracks, deadlines, and random malfunctions.

This assignment uses the Flatland Challenge to progressively assess planning methods across three tasks of increasing complexity. Question 1 involves 40 single-agent instances; Question 2 introduces 56 multi-agent scenarios with collision constraints; and Question 3 extends to larger multi-agent instances with delays and malfunctions.

Performance is evaluated by two metrics. The p score measures average cost per agent:

$$p_{score} = \frac{SIC + T_{max} \cdot \#failed}{\#agents},$$

where SIC is the sum of individual costs and $T_{max} = 8 \cdot (width + height)$ penalises failed agents. In Question 3, a lateness penalty $2 \cdot \#delayed$ is added. The f score normalises student results against staff baselines:

$$f_{score} = \frac{p_{staff}}{p_{score}},$$

with adjustments in Question 3 for both baseline and advanced solutions. Final marks derive from the average f score across all test instances.

This evaluation framework ensures solutions are judged not only on feasibility, but also on efficiency, scalability, and robustness under dynamic conditions, aligning closely with real-world MAPF challenges.

2. Question 1: Warm-up

The warm-up task served as an introduction to the Flatland environment. Each instance consisted of a single agent that must be routed from a start to a target location without considering collisions or temporal constraints, reducing the problem to standard grid-based pathfinding.

I implemented *A* search* with a *Manhattan distance* heuristic, reusing the `lib_piglet` framework for the search engine and open list. The only custom component was a Flatland-specific expander, which generated successor states from the `GridTransitionMap`.

The Manhattan heuristic is admissible and consistent in a 4-connected grid, so A* is both *complete* and *optimal* in this setting. The algorithm's worst-case time and space complexity is $O(b^d)$, where b is the branching factor (the average number of actions available at each state, here at most four rail directions) and d is the solution depth (the number of steps along the optimal path). In other words, in the worst case the number of explored nodes grows exponentially with the path length, although in practice the single-agent instances were trivial.

All 40 test cases were solved optimally in negligible runtime. The sum of individual costs (SIC) matched the makespan in every case, confirming correctness and efficiency. Although not interesting experimentally, this stage confirmed my system's integration with the Flatland API and provided a baseline for extending the approach to time, collisions, and scheduling in subsequent questions.

3. Question 2: Basic Scheduling

The second task extends the warm-up by introducing multiple agents moving simultaneously in the railway network. Each agent must still reach its destination, but now collisions in both space and time must be explicitly avoided: two trains cannot occupy the same cell at the same timestep, nor swap edges in opposite directions. Adding this temporal dimension makes the problem substantially more complex than Question 1. A naïve time-expanded A* would cause an explosion in the state space, so a more efficient approach was needed.

3.1. Safe Interval Path Planning (SIPP)

Safe Interval Path Planning (SIPP) [4] was a natural fit, as it compactly represents each state as a location together with *safe time intervals* when the cell is guaranteed unoccupied. Rather than branching at every timestep, SIPP only considers transitions at the boundaries of safe intervals, which drastically reduces the search space. With an admissible heuristic, SIPP is complete and optimal in the time-extended domain. The search space is $O(|V|\kappa)$ where κ is the average number of safe intervals per cell, compared to $O(|V|t)$ for explicit time expansion.

3.2. First Implementation: Paper-Faithful SIPP

My first implementation followed the SIPP paper's pseudocode closely, constructing full interval lists for every cell and building an edge conflict table. This was a valuable exercise in grounding my understanding of the algorithm, but it was heavy in practice: generating interval lists required iterating across all timesteps and updating occupied times for each agent. The resulting complexity was roughly $O(nt^2)$, with n agents and horizon t. While the approach was correct and guaranteed optimality, performance was limited.

3.3. Second Implementation: Vacancy-Time SIPP

To improve efficiency, I simplified the representation by storing only the *vacancy time* of each node: the first timestep at which its location is no longer guaranteed to be safe, effectively marking the end of its safe interval. Successor generation then "waits until free" before expanding. Conflict checks were rewritten using hashmap lookups, giving constant-time detection for node and edge conflicts. This design avoided splitting intervals entirely while preserving completeness and optimality. Algorithm 1 shows the successor-generation routine.

Algorithm 1 Earliest-Arrival SIPP Expansion (faithful to implementation)

```
function GETSUCCESSORS(s)
    successors ← ∅
    t ← s.time
    v_current ← s.vac_time
    for all valid transitions (loc′, dir′) from s.loc do
        w ← 0                                          ▷ Initialize wait time
        loop
            if t + w ≥ v_current then                  ▷ Cannot wait past current node's vacancy time
                break                                  ▷ Exit inner loop for this transition
            end if
            d ← t + w                                  ▷ Departure time
            a ← d + 1                                  ▷ Arrival time
            if hasNodeConflict(loc′, a) or hasEdgeConflict(s.loc, loc′, d) then
                w ← w + 1                              ▷ Conflict found, wait one more step
                continue                               ▷ Try again with the new wait time
            end if
                                                       ▷ No conflict: found the earliest safe arrival time
            v_new ← getVacTime(loc′, a)
            n ← SIPPNode(loc′, dir′, v_new)
            act ← Action(cost = w + 1)
            add (n, act) to successors
            break                                      ▷ Found the first successor, move to next transition
        end loop
    end for
    return successors
end function
```

In experiments, this version ran about 35% faster than the paper-faithful implementation, while producing identical plans. The speedup came from avoiding upfront interval construction and using indexed lookups instead of scanning lists.

3.4. Reflection

This stage highlighted the danger of reproducing algorithms too literally. My first attempt was faithful but inefficient; the simplified variant, guided by the same theoretical principles, achieved much better performance. By abstracting the core idea of safe intervals into vacancy times, I preserved correctness while gaining efficiency. This adaptation provided the practical backbone needed for scaling to the far more complex multi-agent scenarios with malfunctions and deadlines in Question 3.

4. Question 3: Challenge

The Flatland Challenge is the most demanding stage of the assignment, extending the scheduling problem into a full multi-agent setting with deadlines, delays, and random malfunctions. A valid planner must therefore coordinate collision-free paths for all agents while remaining robust to disruptions and scalable to large instances. Unlike the earlier tasks, no single-agent optimal algorithm suffices here: the challenge requires balancing completeness and optimality guarantees against scalability to hundreds of agents.

To address this, I adopted the approach of Li *et al*. [2], winners of the 2020 Flatland Challenge. Although newer methods exist, their solution was a practical choice under time constraints because it is both well documented and known to scale effectively. Importantly, the paper also analysed why Conflict-Based Search (CBS) [6], a topic heavily discussed in class, was unsuccessful in this domain. CBS is theoretically attractive because it guarantees optimality, but it must resolve conflicts in an exponentially growing joint search tree. This quickly becomes intractable on dense railway networks with hundreds of agents, leading to poor scalability. Based on both the literature and our classroom discussions, I therefore decided not to pursue CBS, focusing instead on an LNS-based approach.

Within this framework, my implementation follows the structure of Li *et al*. but had to be re-engineered for the assignment environment, since their code base and simulator differed significantly. The final system can be decomposed into several key modules, beginning with Large Neighbourhood Search.

4.1. Large Neighbourhood Search (LNS)

LNS [1, 7] is a metaheuristic that iteratively improves a global solution by repeatedly destroying and repairing parts of it. The first step is to construct an *initial solution*, which then serves as the baseline for refinement. The method is heuristic and therefore not complete or optimal in the multi-agent setting, but it is anytime: given more iterations, the quality of the solution improves.

Initial Planning with Prioritised Planning (PP) For initial plans, I used *Prioritised Planning* (PP) [8], in which agents are planned sequentially in a chosen order, with earlier agents treated as dynamic obstacles for later ones. PP is incomplete and order-dependent, but combined with SIPP (see Section 3) it remains efficient and produces valid, collision-free solutions that scale to hundreds of agents.

A key design choice was the heuristic used in SIPP. While Manhattan distance is cheap, it provides weak guidance. I instead adopted a *reverse Dijkstra* heuristic, which precomputes exact goal distances by running a shortest-path search backwards from each target. This yields an admissible and consistent heuristic, ensuring optimal single-agent plans, at the cost of preprocessing. Because PP is reused extensively during LNS, this cost is amortised. In 500 iterations on Level 5, reverse Dijkstra reduced total runtime by more than sixfold compared to Manhattan (Table 1), demonstrating that stronger heuristics are essential for scalability.

I also tested different agent ordering strategies, since ordering directly affects feasibility. Three were evaluated: (1) *random ordering*, a neutral baseline but highly variable; (2) *regret-based ordering*, ranking agents by how much longer their current path is compared to the shortest-path distance; and (3) *slack-based ordering*, prioritising agents with the least deadline slack.

Heuristic	**Total Plan Time (s)**
Manhattan	1403.6
Reverse Dijkstra	232.7

Table 1. Runtime comparison of heuristics in LNS (Level 5, 500 iterations).

Slack-based ordering consistently achieved the lowest p scores (Table 2), making it the best choice for initial planning. Interestingly, later experiments showed that the same strategy performed poorly in local repair (see below), highlighting that different stages of LNS benefit from different priorities.

Prioritisation Strategy	**Total p score**	**Avg. p score**
Random	1492	186.5
Regret-based	1504	188.0
Slack-based	1453	181.6

Table 2. Comparison of agent ordering strategies for initial planning (Level 6).

Neighbourhood Selection The second stage of LNS selects a subgroup of agents (the *neighbourhood*) to replan. After replanning, the new solution is accepted if the SIC improves; using the p score instead worsened performance. Following [1], neighbourhood size was capped at five agents, providing a balance between search space and runtime.

I tested three selection methods: (1) *random walk*, starting from a high-regret agent and adding nearby conflicts; (2) *intersection-based*, choosing agents that passed through a random intersection; and (3) *adaptive LNS*, which maintains weights for each method and samples proportionally to past success [5].

Results after 2500 iterations on Level 6 (Table 3) showed broadly similar performance. Because no fixed method dominated, I adopted the adaptive strategy, which automatically balances between methods for robustness across instances.

Strategy	**Total p score**	**Avg. p score**
Random walk	1303	162.9
Intersection-based	1316	164.5
Adaptive LNS	1305	163.1

Table 3. Comparison of neighbourhood selection strategies (Level 6, 2500 iterations).

To handle agents left unplanned after initial PP, I added

a targeted strategy: seed the neighbourhood with all unplanned agents, then include those whose paths intersect their start locations. These agents were always given priority until planned. Before this change, 27 agents across Levels 0–6 remained unsolved; afterwards, *all* agents were successfully planned.

Repair Phase Once a neighbourhood is chosen, agents must be replanned in some order. Using the same three ordering strategies as before, I found that random ordering surprisingly outperformed regret- and slack-based orderings (Table 4). Unlike initial planning, local repair benefits from diversity rather than deadline-awareness, as randomisation prevents biasing the search toward a subset of agents.

Strategy	Total p score	Avg. p score
Random	1305	163.1
Regret-based	1308	163.5
Slack-based	1344	168.0

Table 4. Repair phase results with different priority strategies (Level 6, 2500 iterations).

4.2. Handling Malfunctions

Minimum Communication Policies (MCP) To address deadlocks caused by malfunctions, I used Minimum Communication Policies (MCP) [3]. MCP enforces a consistent ordering of agents through each cell by associating a queue with every location: only the agent at the head may enter, while others must wait. This ensures deadlock-freedom—unaffected agents continue moving, and blocked agents are delayed locally but do not freeze the system. MCP is therefore complete with respect to avoiding deadlock, but it does not guarantee globally optimal schedules.

A practical challenge was reconciling differences between the published algorithm and the Flatland evaluator. The original MCP assumes a synchronous, step-by-step simulation, while the evaluator fast-forwards to the current timestep whenever a malfunction occurs. I therefore had to carefully restrict changes to the portion of each plan from the current timestep onward. Although this required substantial re-engineering, the resulting system successfully handled malfunctions while maintaining valid global schedules.

Replanning MCP alone prevents deadlock, but queues can cause long local delays. To mitigate this, I applied a final neighbourhood replan after malfunctions. The neighbourhood was seeded with agents whose future paths intersected with malfunctioning trains and expanded to nearby agents likely to be delayed. This targeted destroy-and-repair strategy ensured that only disrupted schedules were modified, preserving efficiency for unaffected agents.

The benefit of this additional step was clear. With MCP alone, the system already outperformed the advanced staff baseline, achieving an f score of 101.35%. Adding the final replan improved performance further to 105.52% (Table 5), showing that local repair complements MCP's safety guarantees with measurable efficiency gains.

Method	f score (%)
MCP only	101.35
MCP + Replanning	105.52

Table 5. Impact of post-malfunction replanning.

4.3. Tuning Maximum Iterations

LNS is an anytime algorithm: solution quality improves with more iterations, but gains eventually plateau. The original Flatland Challenge paper controlled runtime with both an iteration budget and a wall-clock cutoff. For simplicity, I used only a maximum iteration limit and tuned this parameter empirically.

I ran experiments across Levels 0–6 and measured the point at which p score improvements flattened. As shown in Figure 1, Levels 0 and 1 showed no benefit from higher budgets, while Levels 2–4 plateaued relatively early (with Level 2 showing an unreproducible spike at 3000 iterations). By contrast, Levels 5 and 6 continued to improve even beyond 2500 iterations, with Level 5 in particular benefiting from higher limits.

Based on these observations, I reallocated iterations from the easier levels to the harder ones while staying under the two-hour overall runtime cap. The final budgets are listed in Table 6. In practice, the full evaluation completed in 1 hour 46 minutes, indicating that the allocation was slightly conservative but safely within the limit.

Level	Max Iterations
0	100
1	200
2	1200
3	1200
4	2100
5	5500
6	3400

Table 6. Final maximum iteration budgets. More iterations were allocated to harder levels (5 and 6).

4.4. Reflection

Implementing the full LNS framework from scratch required engaging deeply with each component, from heuris-

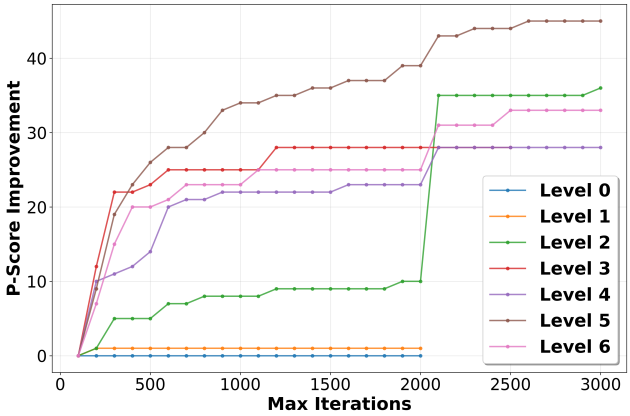

Figure 1. Effect of maximum iterations on p score improvement across Levels 0–6.

tic design to malfunction handling. The experiments reinforced that simple, adaptive strategies (random ordering, adaptive neighbourhoods) often outperformed more structured ones, while preprocessing (reverse Dijkstra) paid off substantially in large instances. The main theoretical compromise was to accept the incompleteness of PP and LNS in exchange for scalability. This trade-off proved effective, allowing my system to outperform the advanced staff baseline. In hindsight, the biggest lesson was that practical robustness often matters more than theoretical guarantees in large-scale MAPF.

5. Conclusion

This assignment required designing a planner capable of coordinating many agents through a railway network under deadlines, malfunctions, and congestion. My solution combined optimal single-agent reasoning (A* and SIPP) with scalable multi-agent heuristics (prioritised planning, LNS, and MCP). This hybrid approach proved highly effective, producing collision-free, deadline-aware schedules for 100% of agents across all test instances. Furthermore, the entire evaluation suite was completed in 1 hour and 46 minutes, well within the two-hour time limit, underscoring the system's efficiency.

The success of this strategy was confirmed by the final competition results, where the system not only outperformed the advanced staff baseline but also secured first place on the leaderboard (Table 7).

This result demonstrates that carefully balancing theoretical guarantees with heuristic scalability can deliver both robustness and high performance in large-scale multi-agent pathfinding problems. Beyond the specific benchmark, the project shows how classical planning principles can be adapted into practical frameworks that handle real-world complexity, highlighting the importance of combining sound theoretical components with metaheuristic strategies to achieve scalable solutions in automated reasoning.

Rank / Submission	Final f score (%)
1. My Submission	**105.52**
2. Runner-up	104.63
3. Third Place	104.26
4. Fourth Place	101.48
Advanced Staff Baseline	*100.00*

Table 7. Final leaderboard results for the Q3 Flatland Challenge.

AI Acknowledgement

In completing this assignment I used generative AI tools in limited, supporting roles. Specifically, AI was used to:

- Assist with implementation by translating pseudocode and algorithm descriptions from research papers into working code.
- Provide help with debugging during development.
- Generate template evaluation scripts for testing different hyperparameter settings.
- Reword and clean sections of this report for clarity, grammar, and readability.

All design decisions, experimental setups, theoretical analyses, and the overall structure and content of the report were my own. The use of AI was supplementary and did not replace my responsibility for understanding, implementing, and critically analysing the methods presented.

References

[1] Jiaoyang Li, Zhe Chen, Daniel Harabor, {Peter J.} Stuckey, and Sven Koenig. Anytime multi-agent path finding via large neighborhood search. In *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence*, pages 4127–4135, United States of America, 2021. Association for the Advancement of Artificial Intelligence (AAAI). Funding Information: We thank Keisuke Okumura for providing us his PPS implementation. The research at the University of Southern California was supported by the National Science Foundation under grant numbers 1409987, 1724392, 1817189, 1837779, and 1935712 as well as a gift from Amazon. The research at Monash University was partially supported by the Australian Research Council under Discovery Grants DP190100013 and DP200100025 as well as a gift from Amazon. Publisher Copyright: © 2021 International Joint Conferences on Artificial Intelligence. All rights reserved.; International Joint Conference on Artificial Intelligence 2021, IJCAI 2021 ; Conference date: 19-08-2021 Through 27-08-2021. 3

[2] Jiaoyang Li, Zhe Chen, Yi Zheng, Shao-Hung Chan, Daniel Harabor, Peter J. Stuckey, Hang Ma, and Sven Koenig. Scalable rail planning and replanning: Winning the 2020 flatland challenge. *Proceedings of the International Conference on Automated Planning and Scheduling*, 31(1):477–485, 2021. 3

[3] Hang Ma, T. K. Satish Kumar, and Sven Koenig. Multi-agent path finding with delay probabilities. *Proceedings of the AAAI Conference on Artificial Intelligence*, 31(1), 2017. 4

[4] Mike Phillips and Maxim Likhachev. Sipp: Safe interval path planning for dynamic environments. In *2011 IEEE International Conference on Robotics and Automation*, pages 5628–5635, 2011. 2

[5] Stefan Ropke and David Pisinger. An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. *Transportation science*, 40(4):455–472, 2006. 3

[6] Guni Sharon, Roni Stern, Ariel Felner, and Nathan R. Sturtevant. Conflict-based search for optimal multi-agent pathfinding. *Artificial Intelligence*, 219:40–66, 2015. 3

[7] Paul Shaw. Using constraint programming and local search methods to solve vehicle routing problems. In *Principles and Practice of Constraint Programming — CP98*, pages 417–431, Berlin, Heidelberg, 1998. Springer Berlin Heidelberg. 3

[8] David Silver. Cooperative pathfinding. *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, 1(1):117–122, 2021. 3